



Client-Server Communication Strategy

CONTENTS

Catalog

01

Strategic Introduction

02

Solving Cross-Origin
Dilemma

03

Production Deployment
and Communication

04

Professional Best Practices



01

Strategic Introduction



Goal of seamless communication

Seamless communication

The goal is to achieve seamless, secure communication between client and server.

Security

Ensuring secure communication is a key aspect of the integration challenge.

Architecture of separated development



Separation of client and server

The architecture involves separating the client (React) and server (Node/Express) in development.



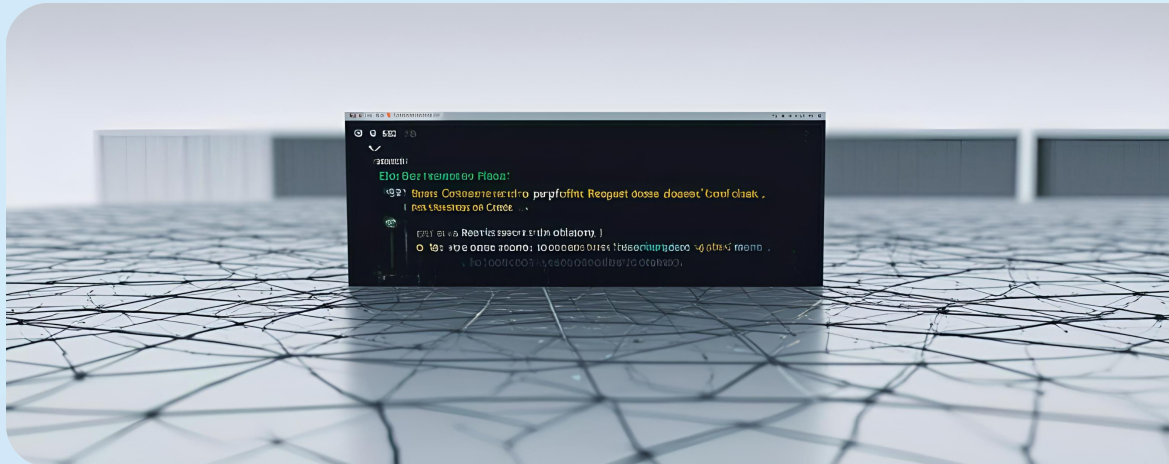
Development efficiency

This separation allows for more efficient development and deployment processes.

Problem of CORS error

Same-Origin Policy

The "Same-Origin Policy" is a security principle that can cause CORS errors.



CORS error

The CORS error occurs when the browser blocks requests due to different origins.



02

Solving Cross-Origin Dilemma

Security principle of Same-Origin Policy



Browser blocks cross-origin requests

The Same-Origin Policy enforces that browsers restrict web pages from making requests to a different domain than the one that served the web page.



Ensuring security and data integrity

This policy is a critical security mechanism that helps to prevent malicious scripts from accessing sensitive data.

Backend solution: CORS Middleware



Explicitly allowing front-end origin

CORS Middleware on the server can be configured to explicitly permit requests from specific client origins.



Avoid wildcard in production

Best practice advises against using the wildcard (*) in production due to security concerns and the need for precise control over allowed origins.

Frontend solution: Proxy setup

Setting up a simple proxy

A proxy can be configured in the React application to make requests appear as if they are from the same origin.

Proxy makes requests Same-Origin

The proxy intercepts requests from the client and forwards them to the server, thus circumventing the Same-Origin Policy restrictions during development.



03

Production Deployment and Communication

Building RESTful Express endpoints



API Contract Definition

Designing RESTful endpoints like `/api/users` to ensure seamless communication.

Express Endpoint Implementation

Building server-side endpoints to handle client requests and responses.

Frontend request making and handling

Fetch or Axios for Requests

Utilizing fetch or Axios libraries to make requests from React.

Asynchronous State Management

Using useState and useEffect to handle async states and errors in React.



Unifying client and server in production



Eliminating Development Proxy

Removing the proxy and serving React build files directly from Express.

Static Files Configuration

Configuring Express to serve the static files of the React application in production.



04

Professional Best Practices

Ensuring type safety and data flow

Type Safety

Ensuring data integrity from the database to the component.

Data Flow Management

Managing data flow to maintain consistency and accuracy.

Security mandate with HTTPS and tokens



Use of HTTPS (TLS)

Securing data in transit with HTTPS (TLS) protocol.



Securing Cookies/Tokens

Protecting sensitive data like cookies and tokens during communication.

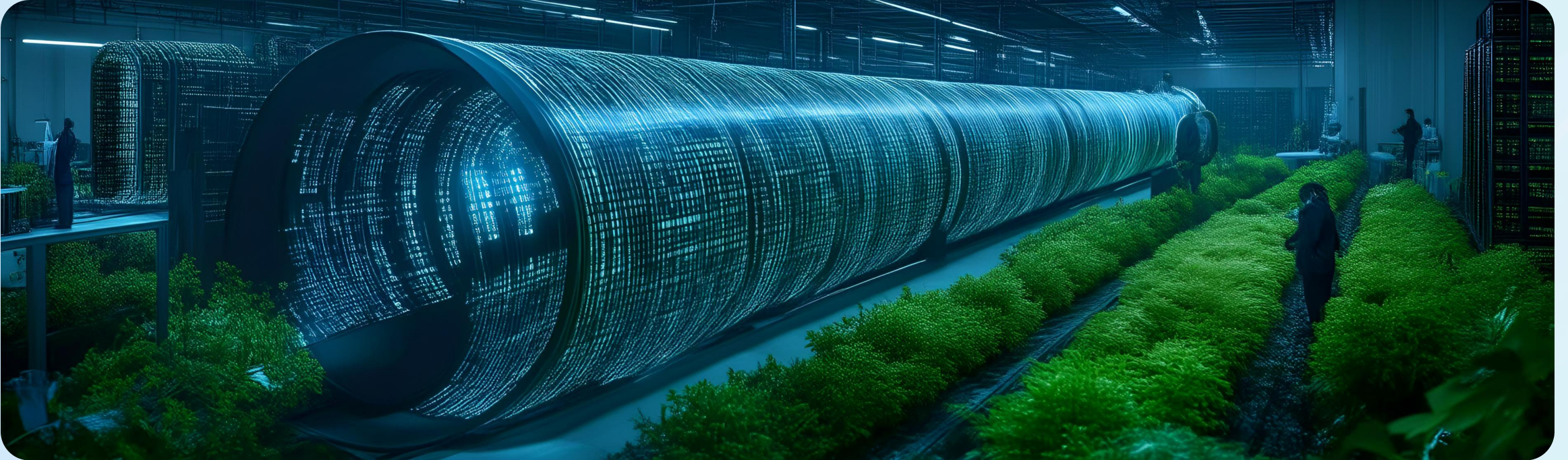
Environment awareness with variables

Environment Variables Usage

Utilizing environment variables to switch between URLs in different environments.



Conclusion on communication pipeline mastery



Essential Communication Pipeline Mastery

Mastery of the communication pipeline is crucial for product deployment.

The background is an abstract composition of various shades of blue and green. It features several overlapping circles of different sizes and colors, ranging from deep blue to light mint green. Interspersed among these circles are numerous thin, straight lines and thicker, pill-shaped strokes in similar color palette. The overall effect is a dynamic, modern, and layered geometric pattern.

THE END

Thank You